

Real-time Neural Signal Decoding System

By Howard Wang, Yibo Wang



Introduction to BCI

1. What is a BCI?

A system that translates neural signals from the brain into control commands for external devices.

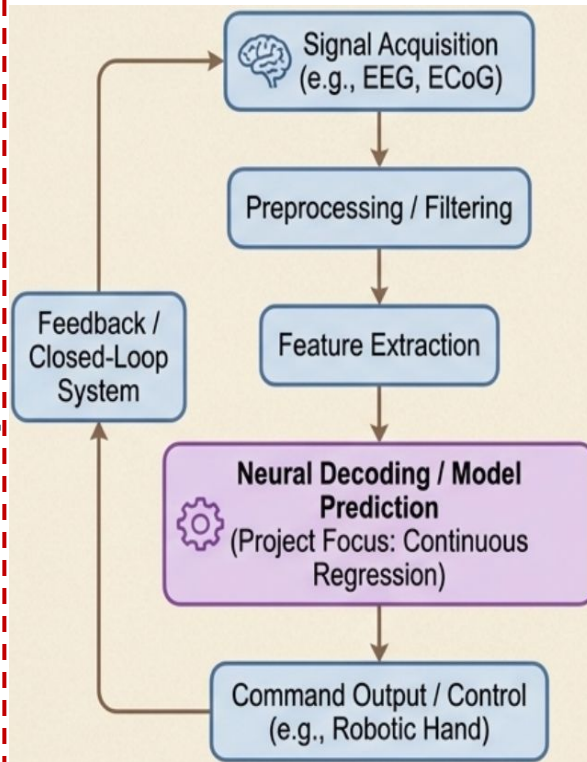
Core Goal: Infer user intent directly from brain activity, bypassing muscles.

Importance: Enables communication/control for paralysis, prosthetics, rehabilitation, and neural engineering.

2. Common Neural Signals

Type	Invasiveness	Resolution
EEG	Non-invasive	Low Spatial
ECoG	Invasive	High Spatial / Temporal
Intracortical	Highly Invasive	Single Neuron

3. Typical BCI Workflow



4. What is Neural Decoding?

Translating patterns of brain activity into meaningful information (intent, movement).

Input: Processed neural features.

Output: Predicted trajectories (e.g., continuous finger positions) or commands.

Challenge: Decoding complex, high-degree-of-freedom fine motor movements.

5. Transition to ECoG for Fine Motor Control

Fine movement decoding requires high-quality, high-frequency signals (high-gamma band).

ECoG provides superior signal quality and spatial resolution compared to scalp EEG for this task.

Trade-off: ECoG is invasive with limited data availability and subject variability.

Why ECoG for Neural Decoding?

ECoG provides the **high-quality, high-frequency** neural data necessary for fine motor decoding, despite inherent collection challenges.

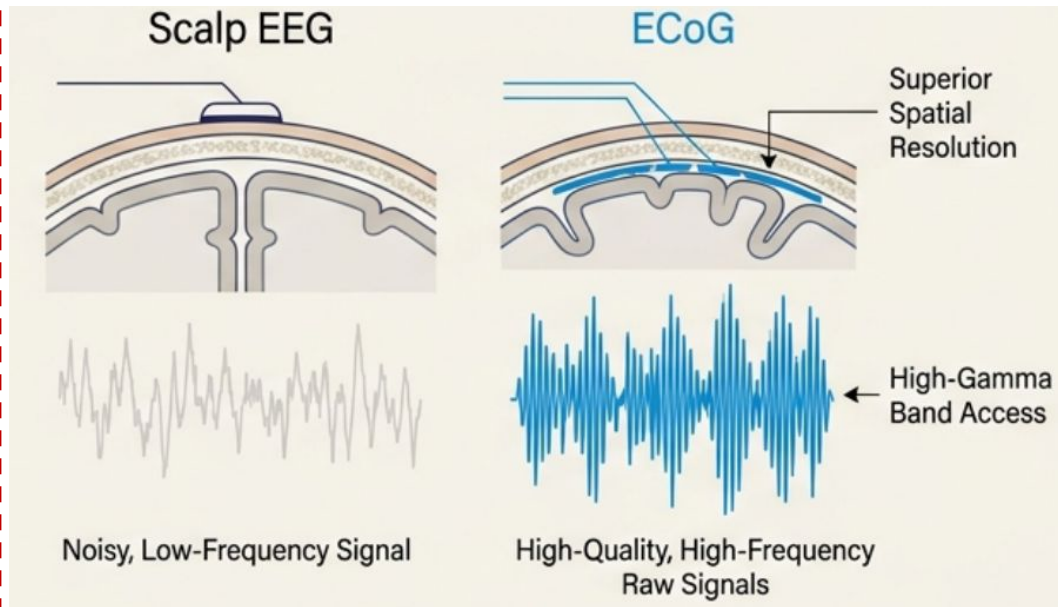
ECoG vs. Scalp EEG: Records directly from the cortical surface, offering superior spatial resolution and lower noise.

Motor Decoding: Crucial for capturing high-frequency brain activity (especially the 'high-gamma band') linked to fine finger movements.

Inherent Limitations: Highly invasive procedure severely limits data availability and sample sizes.

Subject Variability: Neural mapping varies significantly between individuals, requiring adaptable modeling.

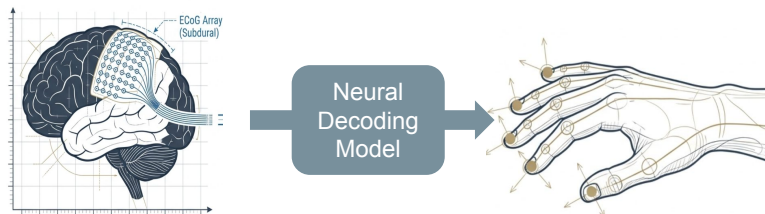
Signal Acquisition & Quality Comparison



Project Overview

Task Summary:

- **Task:** Decode five-finger movement from brain signals.
- **Input:** Recent 300 ms multi-channel ECoG window.
- **Output:** 5D continuous finger movement vector.
- **Update Rate:** One prediction every 50 ms.
- **Nature:** Continuous regression for realtime decoding.



feature-based
regression

Five-Finger
Movement
Prediction

Core Methods:

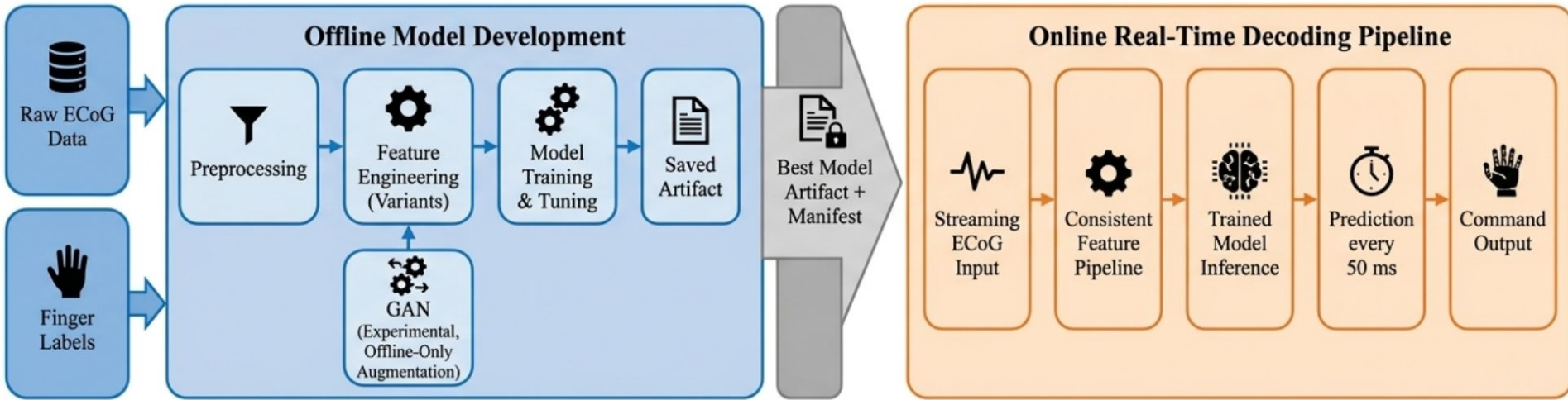
- **Sliding window** over recent neural activity.
- **Label alignment** between ECoG and finger motion.
- **Feature engineering** from raw ECoG signals.
- **Lightweight decoders:** Ridge, MLP, RandomForest, Hoeffding.
- **Stream processing:** sample-by-sample input with buffered window decoding.
- **Stateful decoding:** maintain recent ECoG samples for each session.
- **Low-latency output:** update finger prediction every 50 ms.
- **GAN:** Experimental offline-only **augmentation**, not used in realtime inference.

System Setting:

Offline: model training, tuning, and artifact saving.

Online: streaming ECoG input ➡ buffered decoding ➡ low-latency prediction.

End to End Pipeline



Offline Model Development:

- Uses labeled ECoG data and finger trajectories.
- Performs preprocessing and feature engineering.
- Compares feature variants and lightweight models.
- Tunes models and selects the best artifact.
- GAN is used as offline augmentation.

Artifact Handoff:

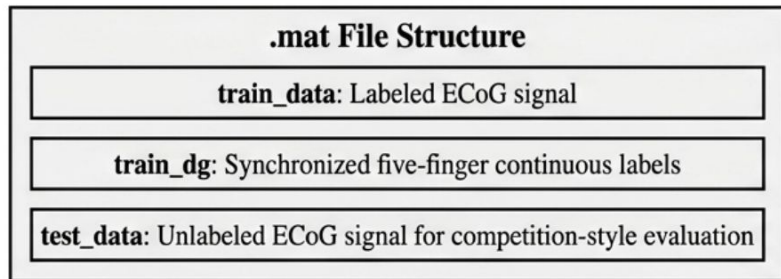
- The best model is saved together with its configuration.
- This ensures training-inference consistency.

Online Real-Time Decoding:

Receives streaming ECoG data.
Applies the same feature pipeline used offline.
Loads the trained model artifact.
Produces low-latency finger predictions every 50 ms.
Sends output to the decoding / control stage.

Data Structure & Temporal Evaluation

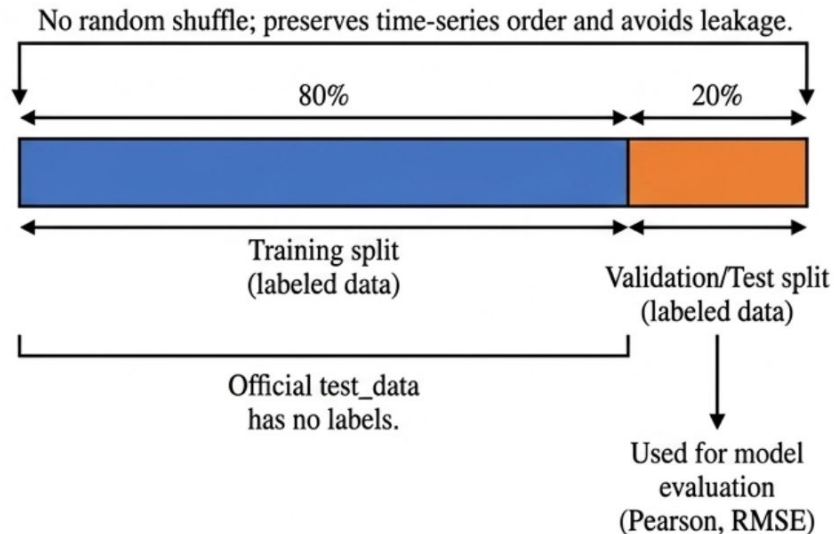
Dataset Structure & Matrix Shapes



Field	Meaning	Example Shape
train_data	multi-channel ECoG signal	(400000, 62) for sub1
train_dg	five-finger continuous labels	(400000, 5)
test_data	unlabeled ECoG signal	(200000, 62) for sub1

- Rows = Time samples, Columns in **train_data** = ECoG channels.
- Columns in **train_dg** = five fingers [thumb, index, middle, ring, little] (continuous vector).
- Sample rate = 1000 Hz.
- Subject channel counts differ (e.g., 48–64 channels).
- **train_dg** is **NOT** a class label; it is a 5D continuous vector.

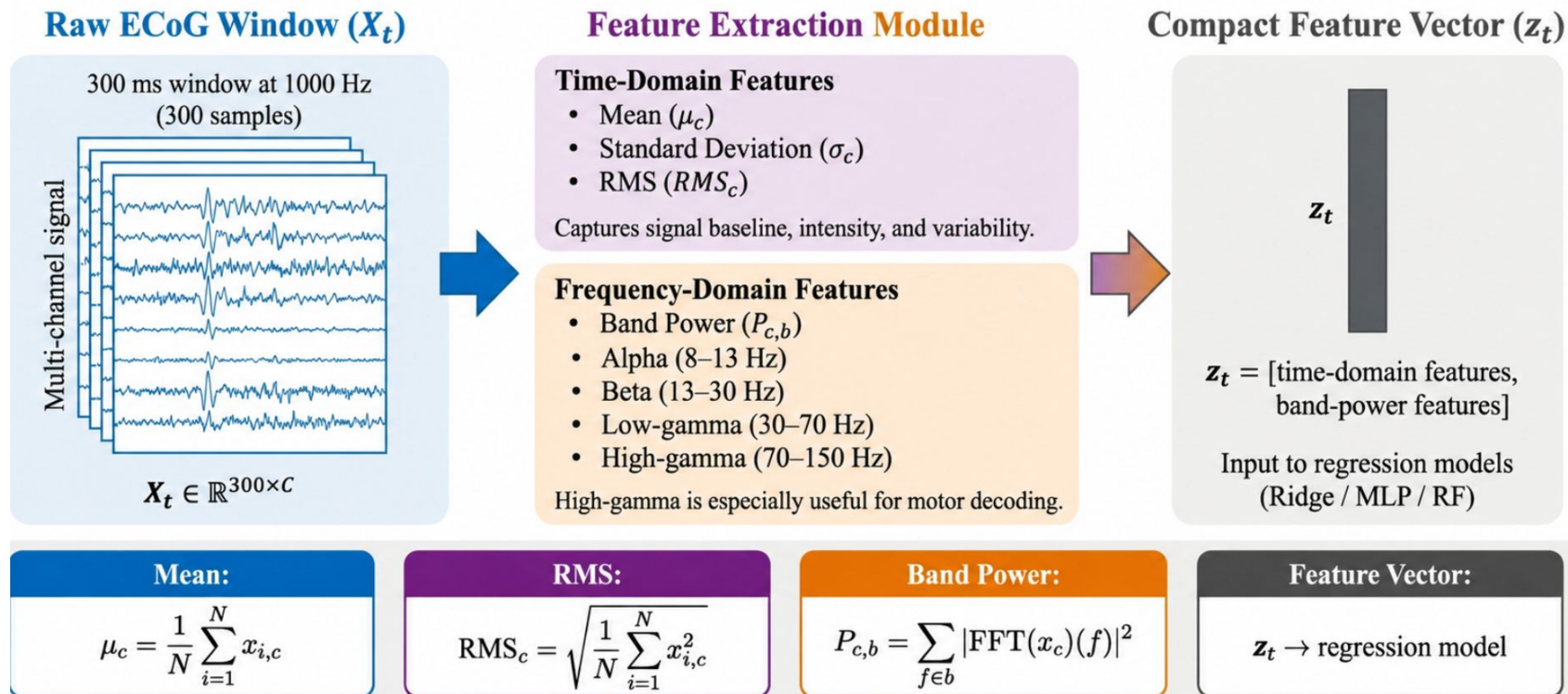
Temporal Split for Evaluation



Evaluation metrics are computed on the held-out labeled validation split, not on the unlabeled competition **test_data**.

Feature Engineering

Feature extraction reduces noise, dimensionality, and inference cost, keeping the decoder lightweight and low-latency.



Feature Engineering

Continuous to Supervised Samples

- Raw ECoG stream is continuous.
- Models need fixed-size supervised samples.
- Sliding windows create many training examples from the timeline.

Sampling & Window Size (1000 Hz)

1 ms	= 1 sample
300 ms Window	= 300 samples
50 ms Step	= 50 samples

Overlapping Windows

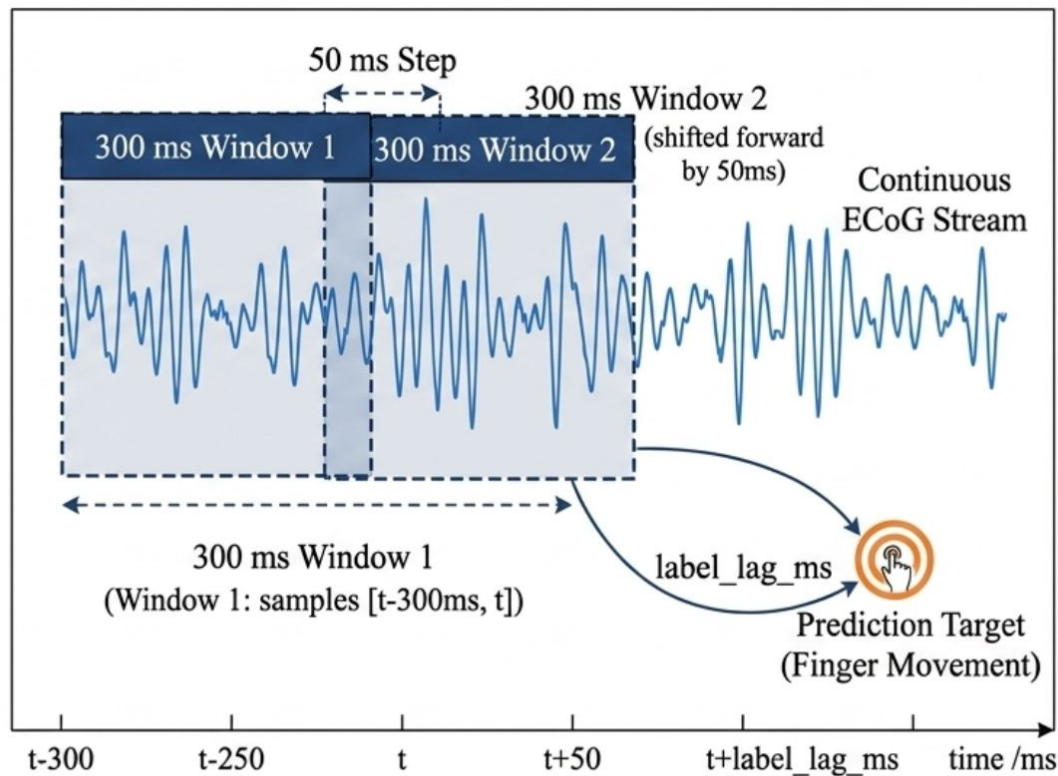
- Heavy overlap allows frequent updates while keeping context.

Delay Alignment (label_lag_ms)

- Finger movement can lag behind neural activity.
- Aligns ECoG window with correct prediction target.
- Lag > 0: Predicts future finger position.

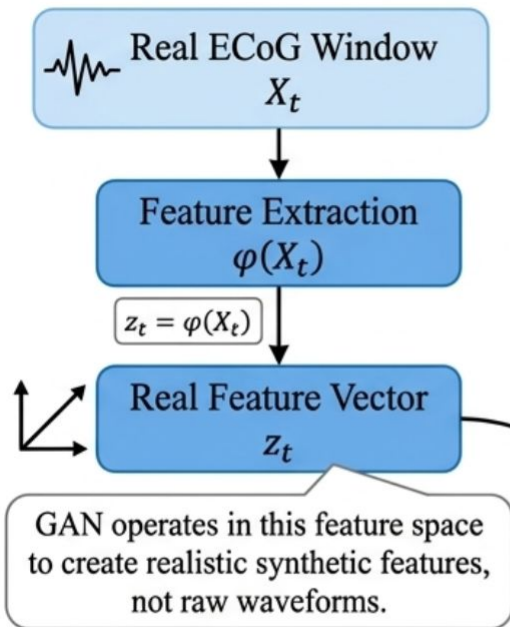
Supervised Formulation

Input ECoG ($X_t \in \mathbb{R}^{300 \times C}$) $\rightarrow$ Target ($y_{t+lag} \in \mathbb{R}^5$)



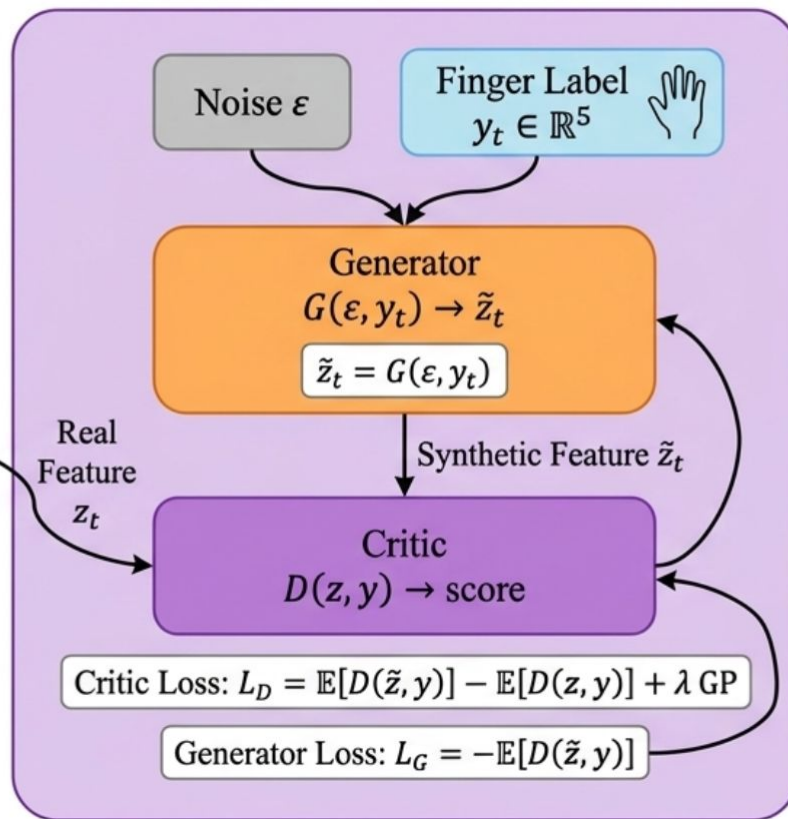
GAN Augmentation

Intuition & Input Connection

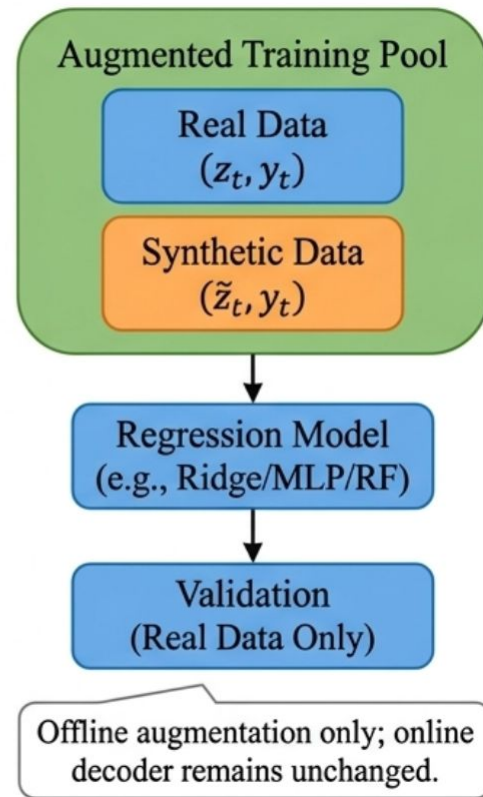


Intuition: Generator creates synthetic features; Critic distinguishes real from synthetic. Generator improves through competition.

Conditional WGAN-GP Architecture



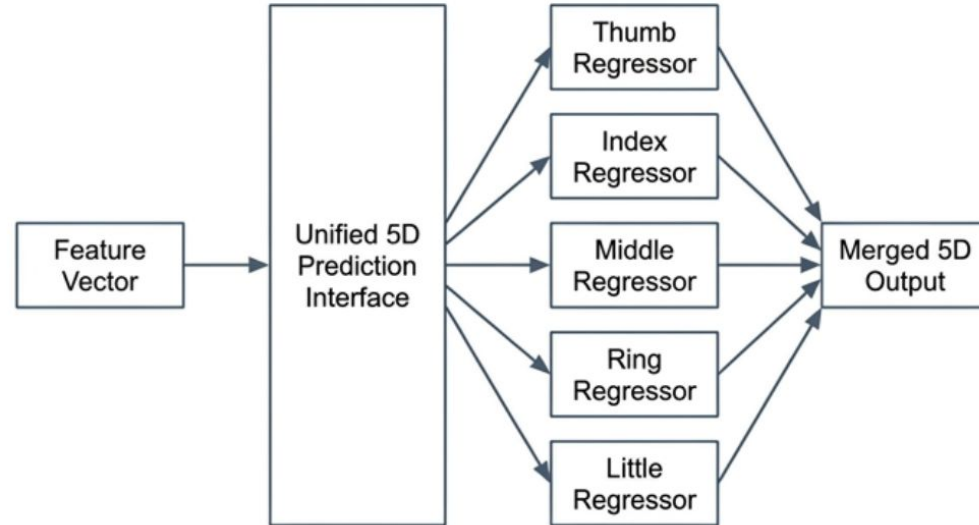
Project Application



Why Lightweight Model

Rationale for Lightweight Architectures

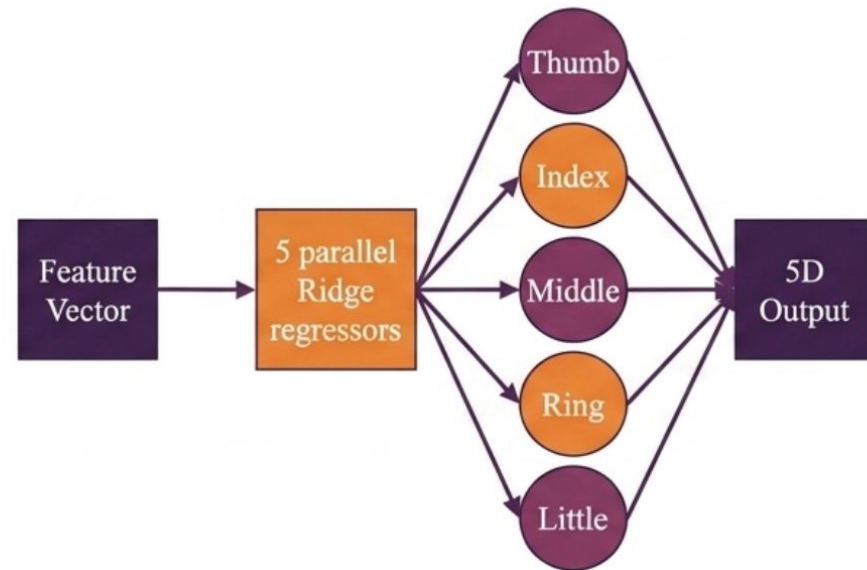
- Prefer Lightweight Models:
 - Low inference latency
 - Simpler deployment
 - Easier debugging and maintenance
 - Better fit for strict 50 ms update requirement
- Why Not Deep Sequence Models:
 - Transformers, LSTMs, CNNs are heavier
 - Higher latency and engineering cost
 - Less suitable for strict realtime decoding
- Model Lineup:
 - Ridge Regression
 - MLP Regressor
 - RandomForest Regressor
 - Hoeffding Tree
- Unified System Design:
 - Externally: Unified 5D prediction output
 - Internally: Ridge and MLP can use separate regressors for each finger



Ridge Regression

• Core Mechanism & Project Role

- Ridge adds L2 regularization to standard linear regression to prevent overfitting and stabilize weights.
- It maps a compact, extracted ECoG feature vector $\mathbf{z}$ to a 5D output $\mathbf{y}$.
- In practice, the wrapper trains independent Ridge regressors for each of the five fingers, ensuring fast, parallel inference.
- The L2 penalty shrinks coefficients towards zero, handling collinear features from the ECoG window.
- **Strengths:** Extremely fast inference, highly stable optimization, interpretable linear baseline, strong deployment reliability.
- **Limitation:** As a linear model, it may miss more complex nonlinear relationships.



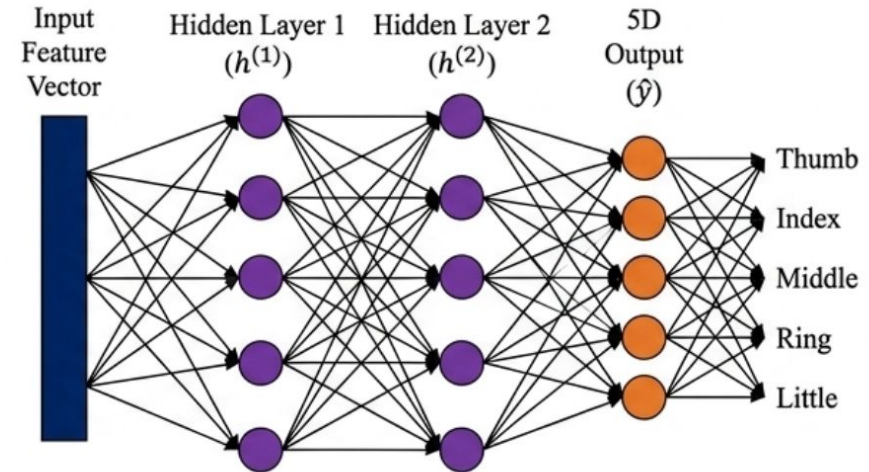
$$\text{Prediction: } \hat{\mathbf{y}} = \mathbf{W}\mathbf{z} + \mathbf{b}$$

$$\text{Ridge Objective: } \min_{\mathbf{W}, \mathbf{b}} \|\mathbf{Y} - (\mathbf{Z}\mathbf{W}^T + \mathbf{b})\|_2^2 + \lambda \|\mathbf{W}\|_2^2$$

MLP

- Core Idea & Project Context

- MLP is a feedforward neural network that introduces nonlinear mapping capabilities through hidden layers and activation functions (σ).
- It transforms the input feature vector from ECoG windows to model complex neural-to-movement patterns that linear baselines like Ridge may miss.
- Output is five continuous predictions for each finger: [Thumb, Index, Middle, Ring, Little].
- Strengths include nonlinear modeling capacity and flexibility, suitable for feature-based input.
- Trade-offs involve requirements for feature scaling, hyperparameter tuning, and higher training costs/latency than Ridge, though it remains significantly lighter than deep sequence models (Transformers, LSTMs).



Hidden Layers: $h^{(1)} = \sigma(W_1 z + b_1)$, $h^{(2)} = \sigma(W_2 h^{(1)} + b_2)$

Output: $\hat{y} = W_o h^{(2)} + b_o$

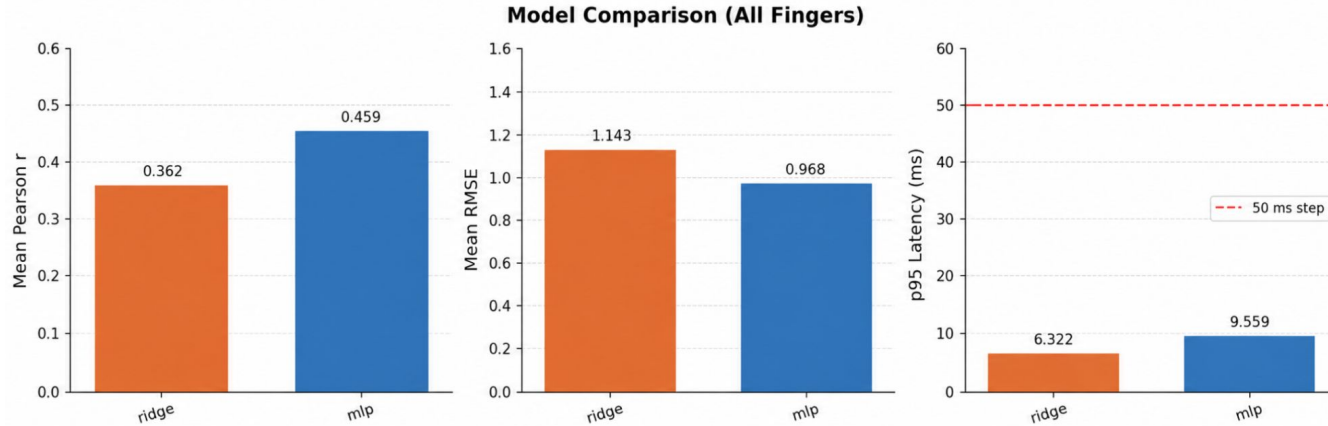
Regression Loss: $L = \|y - \hat{y}\|_2^2$

Other Models

- **Role:** Act as comparative nonlinear baselines.
- **Strengths:** Naturally capture feature interactions and handle unscaled features.
- **Limitations:** Can introduce latency and model size risks compared to Ridge/MLP.
- **Current Focus:** Used for benchmarking, while Ridge (stability) and MLP (nonlinear capacity) remain the primary deployment focus.

Model	Role	Strength	Why Not Main Choice
RandomForest	Nonlinear ensemble baseline	Robust; no strict scaling needs	Larger model files; potential latency risks
Hoeffding Tree	Streaming-style baseline	Conceptually supports incremental learning	Less strong as a primary mainline predictor

Results



Model Comparison (All Fingers)

This figure compares Ridge and MLP using all five fingers.

MLP performs better:

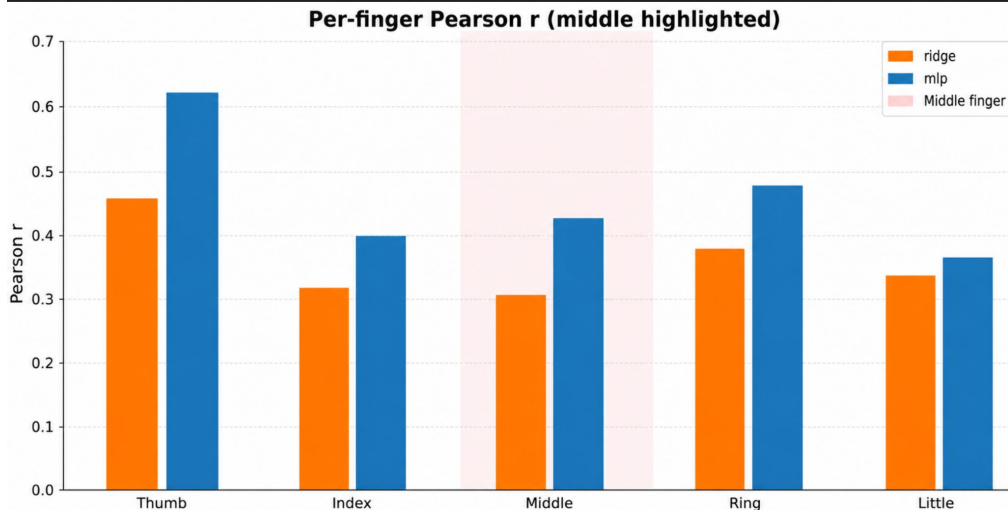
Higher mean Pearson r: 0.459 vs. 0.362

Lower mean RMSE: 0.968 vs. 1.143

Lower p95 latency: 6.322 ms vs. 9.559 ms

This means MLP gives the better accuracy–latency tradeoff for the current feature-based decoding pipeline. Although Ridge is simpler and usually more stable, MLP shows better performance in this run by achieving higher correlation, lower prediction error, and lower latency. Both models are far below the 50 ms real-time decoding step, so the system satisfies the low-latency requirement. The main limitation is still accuracy: a Pearson r around 0.46 indicates moderate decoding performance, not high-precision finger trajectory reconstruction.

Results



- MLP consistently achieves higher Pearson r across all five fingers than Ridge.

- Thumb is the easiest finger to decode in this run, showing the highest Pearson r for both models.

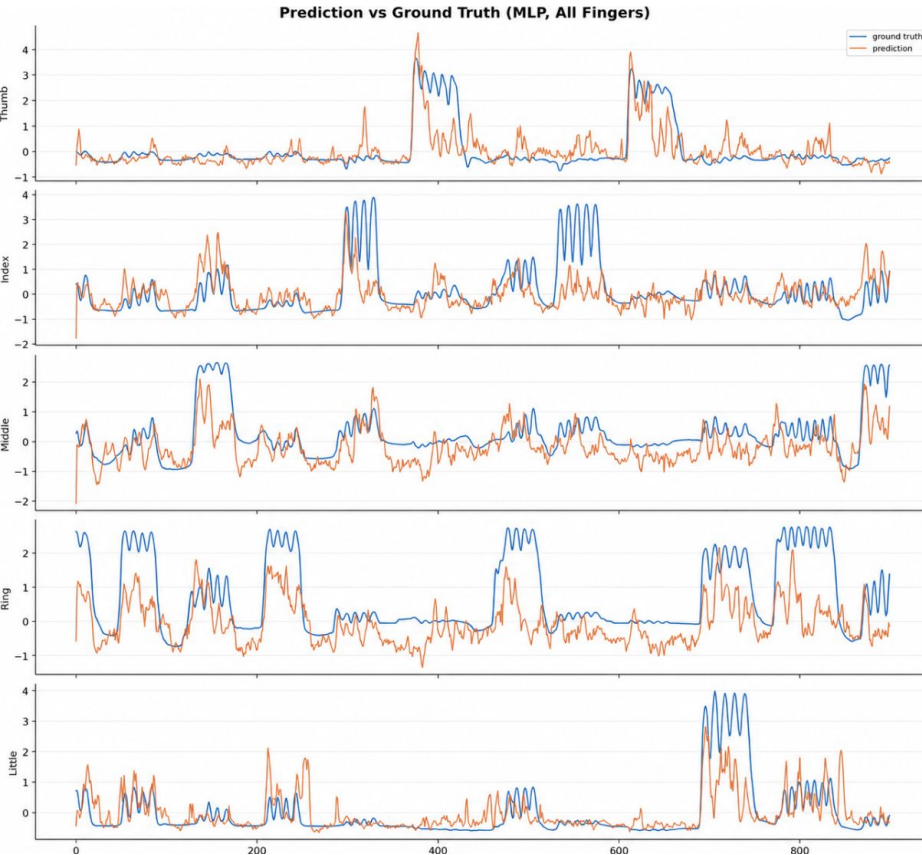
- Index, middle, and little fingers show lower correlations, indicating finger-specific decoding difficulty.

- The middle finger is highlighted here because it remains one of the more challenging fingers and deserves closer inspection.

- Per-finger analysis is important because the overall average can hide weaker finger-level performance.

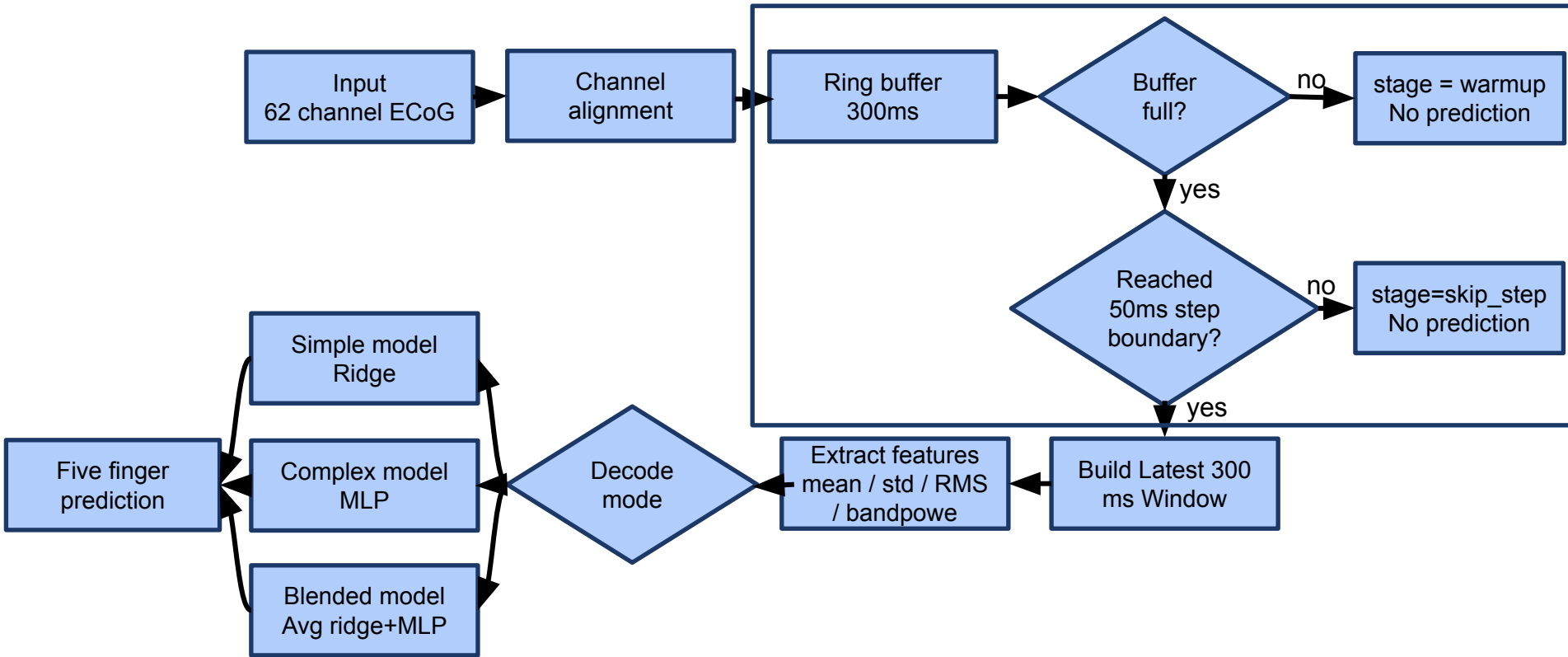
- This result supports using per-finger diagnostics to guide future model improvement and targeted error analysis.

Results



- The MLP decoder captures broad movement trends across several fingers.
- Prediction curves are smoother and often underestimate sharp movement peaks.
- Finger-level behavior varies: some fingers are easier to track than others.
- This confirms moderate decoding accuracy rather than perfect trajectory reconstruction.
- Curve tracking provides a more honest view than aggregate metrics alone.

Pipeline Design



Pipeline Design

Ring buffer:

Size is fixed, moving cursor rather than data

Initial

index	0	1	2	3	...	299
data						

stage = warmup

300 samples
coming in

index	0	1	2	3	...	299
data	s300	s299	s298	s297	...	s1

Trigger
decoding

Replace old data

index	0	1	2	3	...	299
data	s301	s300	s299	s298	...	s2

stage = skip_step
s1

Snap shoot every
50 sample

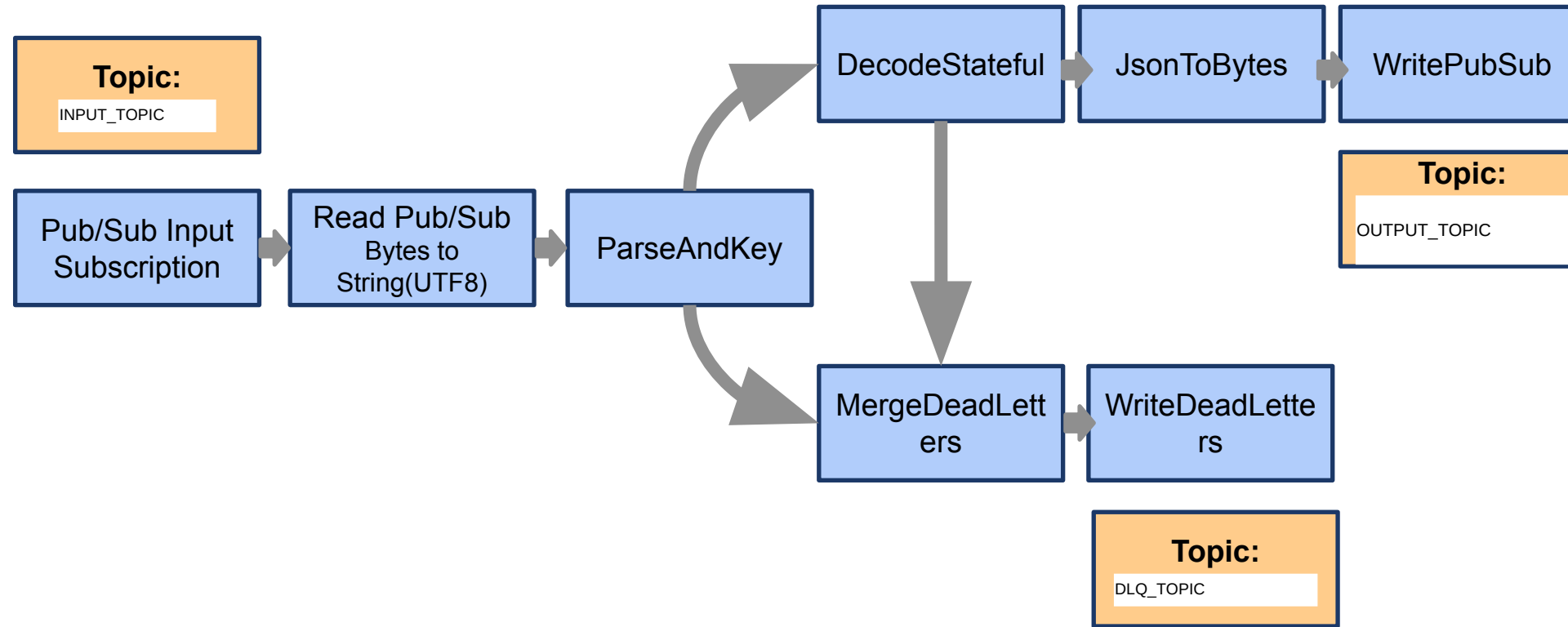
index	0	1	2	3	...	299
data	s350	s349	s248	s247	...	s50

Trigger
decoding

Pipeline Design Trade-Offs

DESIGN	PRO	CON
Not every sample is predicted, decoding is performed once every 50ms	Significantly reduce the number of feature extraction and model prediction computation	Loss the time resolution
Size of the 300ms window	Large window size can provide the model with sufficient context to predict accurately	The computational load during feature extraction will significantly increase
Using blended model	integrating the simple and complex models, the prediction will be more reliable	Two model predictions are required, resulting in a lower throughput
Extract frequency domain features	Enhance the training accuracy	The computational cost of FFT is higher

Dataflow Architecture



Dataflow Configuration

```
cd <PROJECT_ROOT>
python -m src.beam_dataflow `
--runner DataflowRunner `
--project PROJECT_ID `
--region us-east1 `
--no_use_public_ips `
--temp_location "gs://BUCKET /dataflow/tmp" `
--staging_location "gs://BUCKET /dataflow/staging" `
--setup_file "$PWD/setup.py" `
--save_main_session `
--streaming `
--num_workers=8 `
--max_num_workers=8 `
--worker_machine_type=n2-standard-2 `
--input_subscription "projects/PROJECT_ID /subscriptions/INPUT_SUB" `
--output_topic "projects/PROJECT_ID /topics/OUTPUT_TOPIC" `
--deadletter_topic "projects/PROJECT_ID /topics/DLQ_TOPIC" `
--model_simple "gs://BUCKET /model_simple.joblib" `
--model_complex "gs://BUCKET /model_mlp.joblib" `
--mode simple `
--window_ms 300 `
--step_ms 50 `
--legacy_fft_bands `
```

```
python -m src.publish_ecog_stream `
--project PROJECT_ID `
--topic INPUT_TOPIC `
--mat_path ".\data\sub1_comp.mat" `
--segment train `
--session_id bench `
--parallel_sessions 8 `
--target_msg_per_sec 1000`
```

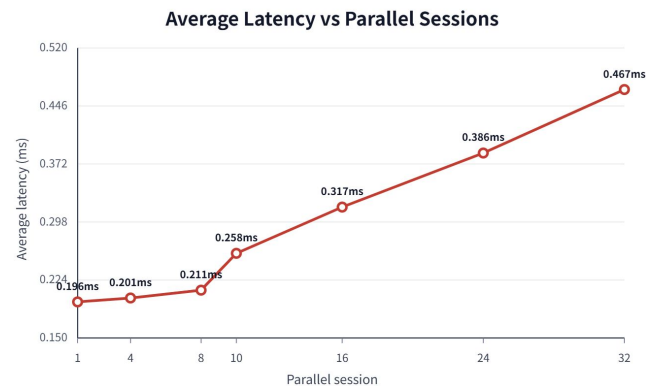
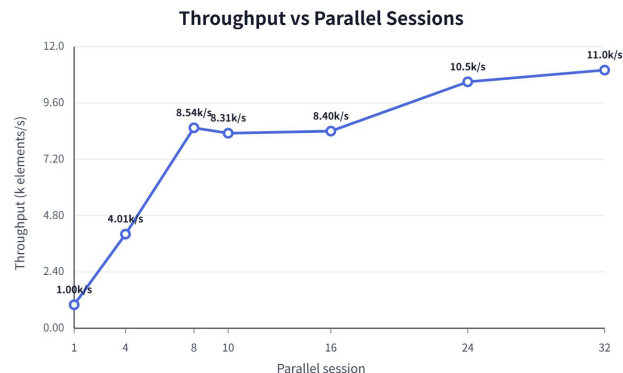
```
cd <PROJECT_ROOT>
python -m src.subscribe_decoded `
--project PROJECT_ID `
--subscription OUTPUT_SUB `
--max_messages 50`
```



Scalable Test

Simple model (Ridge Regression), 8 workers, n2-standard, us-east1

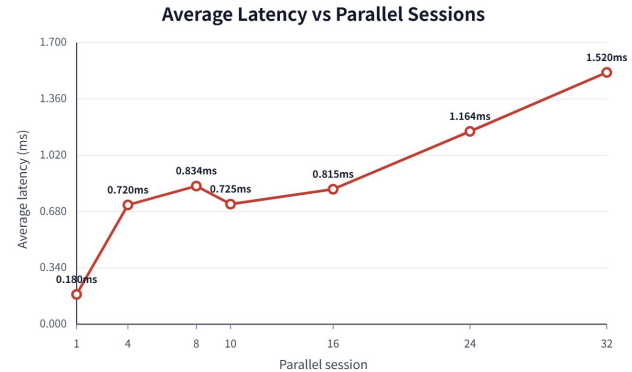
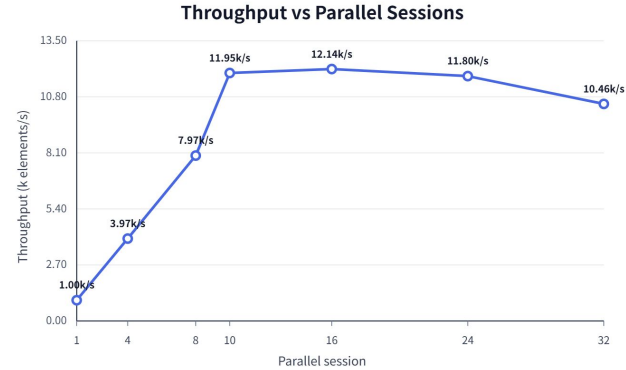
Parallel session	Throughput	Avg Latency
1	1.00k/s	0.196ms
4	4.01k/s	0.201ms
8	8.54k/s	0.211ms
10	8.31k/s	0.258ms
16	8.40k/s	0.317ms
24	10.50k/s	0.386ms
32	11.00k/s	0.467ms



Scalable Test

Complex model (MLP), 8 workers, n2-standard, us-east1

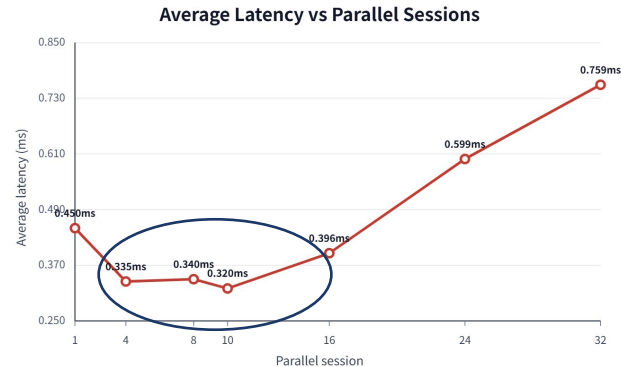
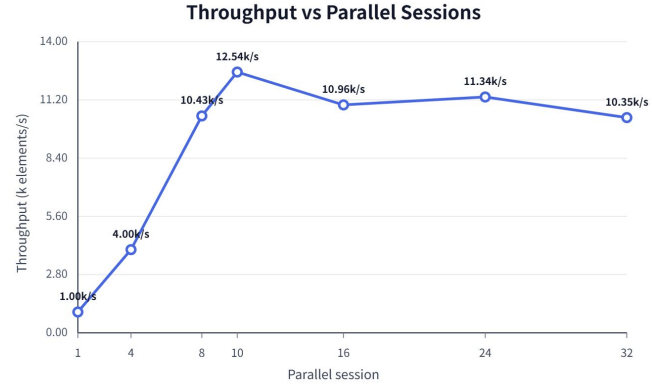
Parallel session	Throughput	Avg Latency
1	1.00k/s	0.180ms
4	3.97k/s	0.720ms
8	7.97k/s	0.834ms
10	11.95k/s	0.725ms
16	12.14k/s	0.815ms
24	11.80k/s	1.164ms
32	10.46k/s	1.520ms



Scalable Test

Blended model (Ridge + MLP), 8 workers, n2-standard, us-east1

Parallel session	Throughput	Avg Latency
1	1.00k/s	0.450ms
4	4.00k/s	0.335ms
8	10.43k/s	0.340ms
10	12.54k/s	0.320ms
16	10.96k/s	0.396ms
24	11.34k/s	0.599ms
32	10.35k/s	0.759ms



Decode data

```
session=bench--p19 t_index=13600 prediction=[-1.1250388189195193, 2.097914558822255, -1.125824254422365, 0.2356300306889761, 5.923337827176254]
session=bench--p19 t_index=13650 prediction=[-1.2577359766970213, 1.252976975344165, -2.130119599792698, 0.30618634987854926, 2.303949034521712]
session=bench--p19 t_index=13700 prediction=[-1.3894612440873366, -0.5687539361857691, -2.793201158688142, 0.27181633283841417, -0.9639138631338445]
session=bench--p19 t_index=13750 prediction=[-1.0658929470122975, -1.5411956974785377, -2.952843434886348, 0.9026788961825838, -2.8236768677568085]
session=bench--p19 t_index=13800 prediction=[-1.3102239394849349, -1.5464695218952158, -4.448239501685972, 1.8176633858103846, -3.440931891141478]
session=bench--p19 t_index=13850 prediction=[-0.9402749067944266, -0.965730500075809, -4.450812015114782, 3.0090917454553856, -1.3914753668096094]
session=bench--p19 t_index=13900 prediction=[-0.6203958743600735, 0.7739539204869574, -4.6639739766221515, 3.6406050337527844, -0.7441758396045426]
session=bench--p19 t_index=13950 prediction=[0.22264102010743234, 3.188540540560703, -3.519409170537162, 3.8390621189034806, 2.4430609049103045]
session=bench--p19 t_index=14000 prediction=[-0.6142472290038536, 6.059911981150049, -2.7362032282990283, 2.4885244225235645, 2.4966579305450773]
```

Five-finger predictions · live

Source: decoded JSON from your Pub/Sub subscription (`prediction` length 5). Finger order: thumb → little.



Future Work

Simulated real-time input, not a real-time signal

Each patient's feature is different, which may require trained model for each people

Accuracy needs to be improved

Complex models like MLP increase latency